DATA SCIENCE LABORATORY ASSIGNMENT 1

Data Wrangling-I: Importing libraries, loading a dataset, preprocessing it, checking for missing values, exploring the dataset, and performing necessary transformations and normalizations to clean the data.

#CODE IMPLEMENTATION

```
# 1. Import required libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# 2. Load dataset
df = pd.read_csv('titanic.csv')   # change path if needed
# 3. Explore dataset
print("First 5 rows:\n", df.head())
print("\nShape of dataset:", df.shape)
print("\nData Types:\n", df.dtypes)
print("\nStatistical Summary:\n", df.describe())
# 4. Check missing values
missing_values = df.isnull().sum()
print("\nMissing Values:\n", missing_values)
# 5. Data preprocessing
# Convert Age to numeric
df['Age'] = pd.to_numeric(df['Age'], errors='coerce')
# Convert Survived to categorical
df['Survived'] = df['Survived'].astype('category')
# 6. Handle missing values
# Fill Age with mean
df['Age']=df['Age'].fillna(df['Age'].mean())
# 7. Normalize numerical columns
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler()
```

df[['Age', 'Fare']] = scaler.fit_transform(df[['Age', 'Fare']])

 # 8. Convert categorical to numerical

 # Convert Sex column

df['Sex'] = df['Sex'].map({'male': 0, 'female': 1})

 # One-hot encoding for Embarked

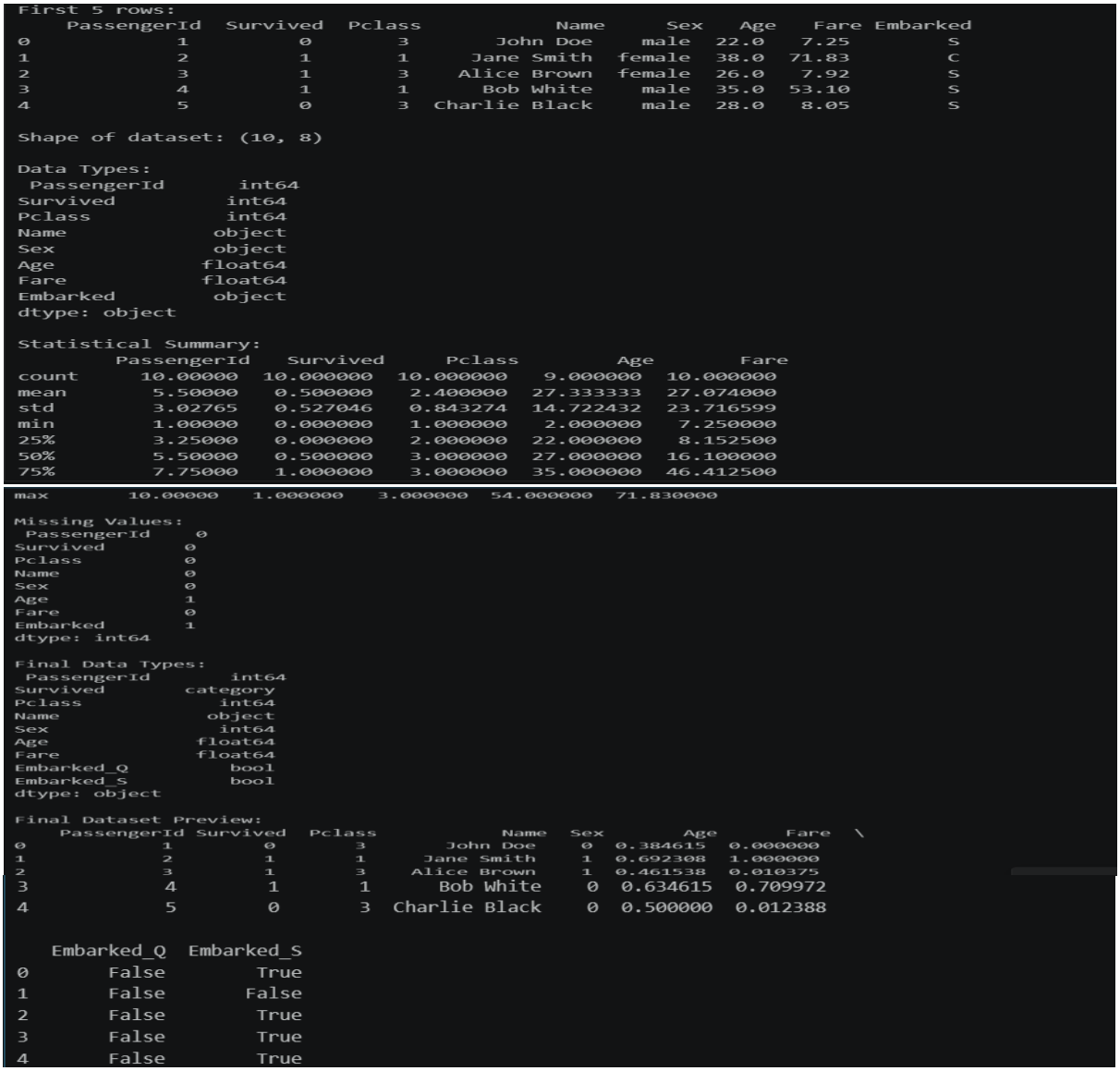
df = pd.get_dummies(df, columns=['Embarked'], drop_first=True)

 # 9. Check final dataset

print("\nFinal Data Types:\n", df.dtypes)

print("\nFinal Dataset Preview:\n", df.head())

OUTPUT:



```
First 5 rows:
   PassengerId  Survived  Pclass            Name     Sex   Age    Fare Embarked
0            1         0       3        John Doe    male  22.0    7.25        S
1            2         1       1      Jane Smith  female  38.0   71.83        C
2            3         1       3     Alice Brown  female  26.0    7.92        S
3            4         1       1       Bob White    male  35.0   53.10        S
4            5         0       3   Charlie Black    male  28.0    8.05        S

Shape of dataset: (10, 8)

Data Types:
 PassengerId      int64
Survived          int64
Pclass            int64
Name             object
Sex              object
Age             float64
Fare            float64
Embarked         object
dtype: object

Statistical Summary:
       PassengerId    Survived      Pclass         Age        Fare
count     10.00000  10.000000  10.000000    9.000000  10.000000
mean       5.50000   0.500000   2.400000   27.333333  27.074000
std        3.02765   0.527046   0.843274   14.722432  23.716599
min        1.00000   0.000000   1.000000    2.000000   7.250000
25%        3.25000   0.000000   2.000000   22.000000   8.152500
50%        5.50000   0.500000   3.000000   27.000000  16.100000
75%        7.75000   1.000000   3.000000   35.000000  46.412500
max       10.00000   1.000000   3.000000   54.000000  71.830000

Missing Values:
 PassengerId    0
Survived        0
Pclass          0
Name            0
Sex             0
Age             1
Fare            0
Embarked        1
dtype: int64

Final Data Types:
 PassengerId       int64
Survived        category
Pclass             int64
Name              object
Sex                int64
Age              float64
Fare             float64
Embarked_Q          bool
Embarked_S          bool
dtype: object

Final Dataset Preview:
   PassengerId Survived  Pclass            Name  Sex       Age       Fare  \
0            1        0       3        John Doe    0  0.384615  0.000000
1            2        1       1      Jane Smith    1  0.692308  1.000000
2            3        1       3     Alice Brown    1  0.461538  0.010375
3            4        1       1       Bob White    0  0.634615  0.709972
4            5        0       3   Charlie Black    0  0.500000  0.012388

   Embarked_Q  Embarked_S
0       False        True
1       False       False
2       False        True
3       False        True
4       False        True
```

DATA SCIENCE LABORATORY ASSIGNMENT 2

Data Wrangling-II: Create a synthetic "Academic Performance" dataset for students, and then perform various data wrangling operations in Python. The tasks will involve: 1. Scanning the dataset for missing values and inconsistencies, and handling them. 2. Identifying and dealing with outliers in numeric variables. 3. Applying data transformations on at least one variable for better understanding or to improve distribution.

 # #CODE IMPLEMENTATION

 # 1. Import Libraries

```
import pandas as pd
import numpy as np
```

 # 2. Create Synthetic Academic Dataset

```
data = {
    'Student ID': range(1, 101),
    'Age': np.random.randint(18, 25, 100),
    'Gender': np.random.choice(['Male', 'Female'], 100),
    'Subject 1': np.random.randint(50, 100, 100),
    'Subject 2': np.random.randint(60, 90, 100),
    'Subject 3': np.random.randint(55, 95, 100),
    'Attendance': np.random.uniform(70, 100, 100),
    'Final Grade': np.random.choice(['A', 'B', 'C'], 100)
}
df = pd.DataFrame(data)
```

 # 3. Introduce Missing Values

```
df.loc[5, 'Subject 1'] = np.nan
df.loc[20, 'Subject 2'] = np.nan
df.loc[35, 'Attendance'] = np.nan
```

 # 4. Check Missing Values

```
print("Missing Values:\n", df.isnull().sum())
```

 # 5. Handle Missing Values

```
df['Subject 1'] = df['Subject 1'].fillna(df['Subject 1'].mean())
df['Subject 2'] = df['Subject 2'].fillna(df['Subject 2'].median())
df['Attendance'] = df['Attendance'].fillna(df['Attendance'].mean())
```

```
# 6. Handle Inconsistencies
df['Attendance'] = df['Attendance'].clip(0, 100)
df['Final Grade'] = df['Final Grade'].apply(
    lambda x: x if x in ['A', 'B', 'C'] else 'C'
)
print("\nMissing Values After Cleaning:\n", df.isnull().sum())
# 7. Function to Detect Outliers (IQR)
def detect_outliers(df, column):
    Q1 = df[column].quantile(0.25)
    Q3 = df[column].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    return df[(df[column] < lower_bound) | (df[column] > upper_bound)]
# 8. Detect Outliers
print("\nOutliers in Subject 1:\n", detect_outliers(df, 'Subject 1'))
print("\nOutliers in Subject 2:\n", detect_outliers(df, 'Subject 2'))
# 9. Handle Outliers using Clipping
for col in ['Subject 1', 'Subject 2', 'Subject 3', 'Attendance']:
    Q1 = df[col].quantile(0.25)
    Q3 = df[col].quantile(0.75)
    IQR = Q3 - Q1
    lower = Q1 - 1.5 * IQR
    upper = Q3 + 1.5 * IQR
    df[col] = np.clip(df[col], lower, upper)
# 10. Check Skewness Before Transformation
print("\nSkewness before:", df['Subject 1'].skew())
# 11. Apply Log Transformation
df['Subject 1'] = np.log(df['Subject 1'] + 1)
# 12. Check Skewness After Transformation
print("Skewness after:", df['Subject 1'].skew())
```

 # 13. Final Dataset Preview

print("\nFinal Dataset:\n", df.head())

OUTPUT:

```
Missing Values:
 Student ID     0
Age             0
Gender          0
Subject 1       1
Subject 2       1
Subject 3       0
Attendance      1
Final Grade     0
dtype: int64

Missing Values After Cleaning:
 Student ID     0
Age             0
Gender          0
Subject 1       0
Subject 2       0
Subject 3       0
Attendance      0
Final Grade     0
dtype: int64

Outliers in Subject 1:
 Empty DataFrame
Columns: [Student ID, Age, Gender, Subject 1, Subject 2, Subject 3, Attendance, Final Grade]
Index: []

Outliers in Subject 2:
 Empty DataFrame
Columns: [Student ID, Age, Gender, Subject 1, Subject 2, Subject 3, Attendance, Final Grade]
Columns: [Student ID, Age, Gender, Subject 1, Subject 2, Subject 3, Attendance, Final Grade]
Index: []

Skewness before: 0.09060079784458387
Skewness after: -0.18486121430013353

Final Dataset:
    Student ID  Age  Gender  Subject 1  Subject 2  Subject 3  Attendance  \
0           1   21  Female   4.532599       70.0         64   88.877992
1           2   21  Female   4.262680       67.0         70   93.066579
2           3   18  Female   3.951244       77.0         92   78.226383
3           4   18    Male   4.343805       65.0         57   86.343441
4           5   19    Male   4.553877       61.0         90   99.103970

  Final Grade
0           C
1           C
2           B
3           C
4           A
```

DATA SCIENCE LABORATORY ASSIGNMENT 3

Descriptive Statistics - Measures of Central Tendency and variability: Using the Iris dataset. We will: a) Perform operations on the Iris dataset to calculate basic statistical details (percentile, mean, standard deviation, etc.) for different species ('Iris-setosa', 'Iris-versicolor', 'Iris virginica'). b) Use Python to extract and analyze these statistics for each species and understand their distributions.

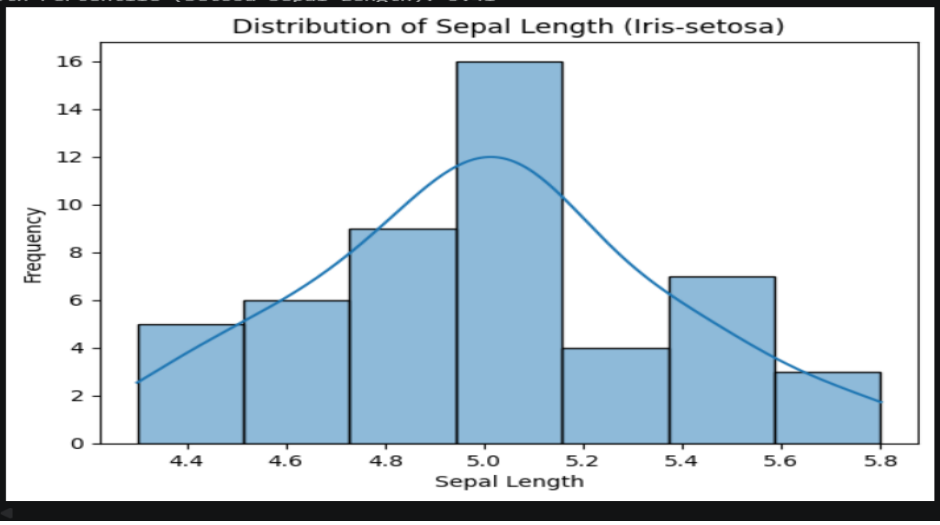
#CODE IMPLEMENTATION

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
# 2. Load Dataset
df = pd.read_csv('/iris_dataset.csv')
print("First 5 rows:\n", df.head())
# 3. Filter by Species
setosa = df[df['Species'] == 'setosa']
versicolor = df[df['Species'] == 'versicolor']
virginica = df[df['Species'] == 'virginica']
# 4. Descriptive Statistics
print("\nIris-setosa Statistics:\n", setosa.describe())
print("\nIris-versicolor Statistics:\n", versicolor.describe())
print("\nIris-virginica Statistics:\n", virginica.describe())
# 5. Percentile Calculation
p10 = np.percentile(setosa['SepalLength'], 10)
p90 = np.percentile(setosa['SepalLength'], 90)
print("\n10th Percentile (Setosa Sepal Length):", p10)
print("90th Percentile (Setosa Sepal Length):", p90)
# 6. Visualization
sns.histplot(setosa['SepalLength'], kde=True)
plt.title('Distribution of Sepal Length (Iris-setosa)')
plt.xlabel('Sepal Length')
plt.ylabel('Frequency')
```

plt.show()

OUTPUT:

```
First 5 rows:
   SepalLength  SepalWidth  PetalLength  PetalWidth Species
0          5.1         3.5          1.4         0.2  setosa
1          4.9         3.0          1.4         0.2  setosa
2          4.7         3.2          1.3         0.2  setosa
3          4.6         3.1          1.5         0.2  setosa
4          5.0         3.6          1.4         0.2  setosa

Iris-setosa Statistics:
       SepalLength  SepalWidth  PetalLength  PetalWidth
count     50.00000   50.000000    50.000000   50.000000
mean       5.00600    3.428000     1.462000    0.246000
std        0.35249    0.379064     0.173664    0.105386
min        4.30000    2.300000     1.000000    0.100000
25%        4.80000    3.200000     1.400000    0.200000
50%        5.00000    3.400000     1.500000    0.200000
75%        5.20000    3.675000     1.575000    0.300000
max        5.80000    4.400000     1.900000    0.600000

Iris-versicolor Statistics:
       SepalLength  SepalWidth  PetalLength  PetalWidth
count    50.000000   50.000000    50.000000   50.000000
mean      5.936000    2.770000     4.260000    1.326000
std       0.516171    0.313798     0.469911    0.197753
min       4.900000    2.000000     3.000000    1.000000
25%       5.600000    2.525000     4.000000    1.200000
50%       5.900000    2.800000     4.350000    1.300000
75%       6.300000    3.000000     4.600000    1.500000
max       7.000000    3.400000     5.100000    1.800000

Iris-virginica Statistics:
       SepalLength  SepalWidth  PetalLength  PetalWidth
count     50.00000   50.000000    50.000000    50.00000
mean       6.58800    2.974000     5.552000     2.02600
std        0.63588    0.322497     0.551895     0.27465
min        4.90000    2.200000     4.500000     1.40000
25%        6.22500    2.800000     5.100000     1.80000
50%        6.50000    3.000000     5.550000     2.00000
75%        6.90000    3.175000     5.875000     2.30000
max        7.90000    3.800000     6.900000     2.50000

10th Percentile (Setosa Sepal Length): 4.59
90th Percentile (Setosa Sepal Length): 5.41
```



DATA SCIENCE LABORATORY ASSIGNMENT 4

Data Analytics-I: Create a Linear Regression Model using Python/R to predict home prices using Boston Housing Dataset. The Boston Housing dataset contains information about various houses in Boston through different parameters. There are 506 samples and 14 feature variables in this dataset. The objective is to predict the value of prices of the house using the given features

#CODE IMPLEMENTATION

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
# 2. Load Dataset
url = 'https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.csv'
columns = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX', 'RM', 'AGE',
        'DIS', 'RAD', 'TAX', 'PTRATIO', 'B', 'LSTAT', 'MEDV']
df = pd.read_csv(url, names=columns)
print("First 5 rows:\n", df.head())
print("\nMissing Values:\n", df.isnull().sum())
# 3. Data Visualization
plt.figure(figsize=(10,8))
sns.heatmap(df.corr(), annot=True, cmap='coolwarm', fmt='.2f')
plt.title("Correlation Matrix")
plt.show()
# 4. Prepare Data
X = df.drop('MEDV', axis=1)
y = df['MEDV']
X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.2, random_state=42
)
```

 # 5. Train Model

```
model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)
```

 # 6. Evaluate Model

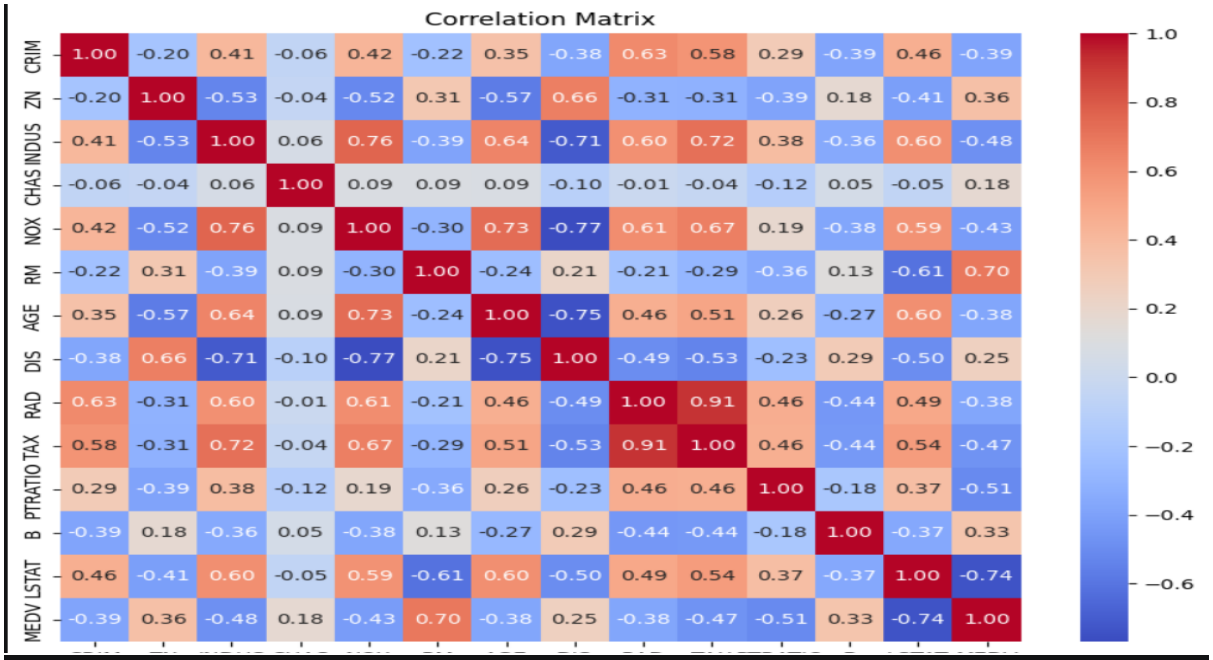
```
mse = mean_squared_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
print("\nMean Squared Error:", mse)
print("R-squared:", r2)
```

 # 7. Visualization (Actual vs Predicted)

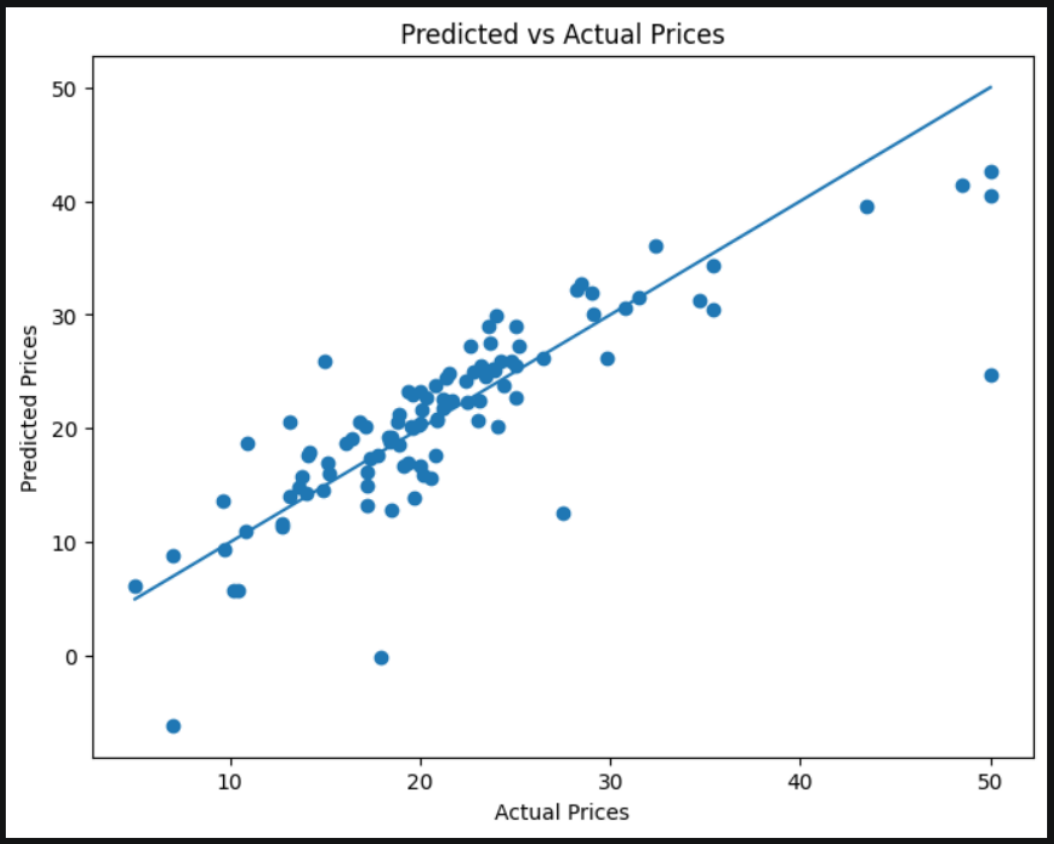
```
plt.figure(figsize=(8,6))
plt.scatter(y_test, y_pred)
plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()])
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Predicted vs Actual Prices")
plt.show()
```

OUTPUT:

```
      CRIM    ZN  INDUS  CHAS    NOX     RM   AGE     DIS  RAD    TAX  \
0  0.00632  18.0   2.31     0  0.538  6.575  65.2  4.0900    1  296.0
1  0.02731   0.0   7.07     0  0.469  6.421  78.9  4.9671    2  242.0
2  0.02729   0.0   7.07     0  0.469  7.185  61.1  4.9671    2  242.0
3  0.03237   0.0   2.18     0  0.458  6.998  45.8  6.0622    3  222.0
4  0.06905   0.0   2.18     0  0.458  7.147  54.2  6.0622    3  222.0

   PTRATIO       B  LSTAT  MEDV
0     15.3  396.90   4.98  24.0
1     17.8  396.90   9.14  21.6
2     17.8  392.83   4.03  34.7
3     18.7  394.63   2.94  33.4
4     18.7  396.90   5.33  36.2

Missing Values:
 CRIM       0
ZN          0
INDUS       0
CHAS        0
NOX         0
RM          0
AGE         0
DIS         0
RAD         0
TAX         0
PTRATIO     0
B           0
LSTAT       0
MEDV        0
dtype: int64
```




```
Mean Squared Error: 24.291119474973534
R-squared: 0.6687594935356318
```



DATA SCIENCE LABORATORY ASSIGNMENT 5

Data Analytics-II 1. Implement logistic regression using Python/R to perform classification on Social_Network_Ads.csv dataset. 2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall on the given dataset.

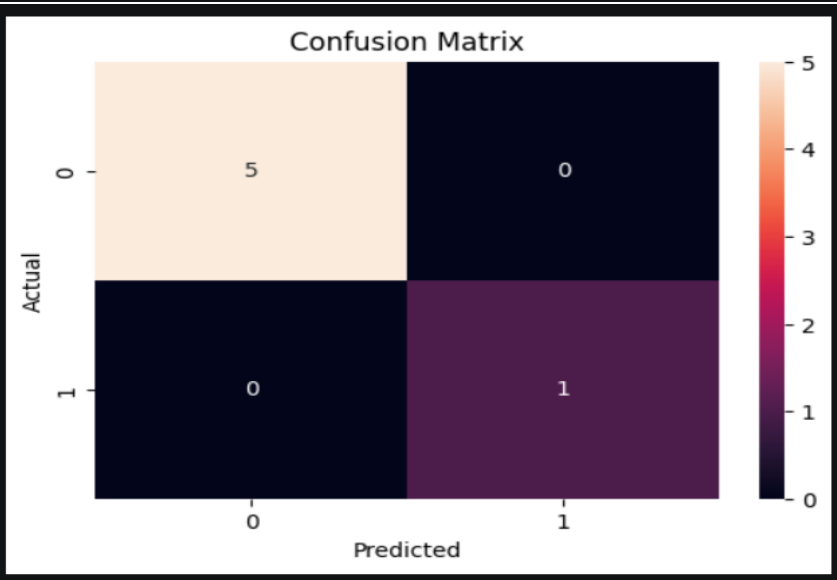
#CODE IMPLEMENTATION

```
# 1. Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
# 2. Load Dataset
dataset = pd.read_csv('Social_Network_Ads.csv')
print("First 5 rows:\n", dataset.head())
print("\nMissing Values:\n", dataset.isnull().sum())
# 3. Preprocessing
# Features (Age, EstimatedSalary)
X = dataset.iloc[:, 2:-1].values
# Target (Purchased)
y = dataset.iloc[:, -1].values
# 4. Train-Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
# 5. Feature Scaling
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# 6. Train Logistic Model
model = LogisticRegression(random_state=42)
model.fit(X_train, y_train)
# 7. Prediction
y_pred = model.predict(X_test)
# 8. Confusion Matrix
cm = confusion_matrix(y_test, y_pred)
TN, FP, FN, TP = cm.ravel()
print("\nConfusion Matrix:\n", cm)
print(f"\nTP: {TP}, TN: {TN}, FP: {FP}, FN: {FN}")
# 9. Performance Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
error_rate = 1 - accuracy
print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Error Rate:", error_rate)
# 10. Confusion Matrix Visualization
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d')
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.show()
```

OUTPUT:

```
First 5 rows:
     User ID  Gender  Age  EstimatedSalary  Purchased
0  15624510    Male   19            19000          0
1  15810944    Male   35            20000          0
2  15668575  Female   26            43000          0
3  15603246  Female   27            57000          0
4  15804002    Male   19            76000          0

Missing Values:
 User ID            0
Gender              0
Age                 0
EstimatedSalary     0
Purchased           0
dtype: int64

Confusion Matrix:
 [[5 0]
 [0 1]]

TP: 1, TN: 5, FP: 0, FN: 0

Accuracy: 1.0
Precision: 1.0
Recall: 1.0
Error Rate: 0.0
```



DATA SCIENCE LABORATORY ASSIGNMENT 6

Data Analytics-III 1. Implement Simple Naïve Bayes classification algorithm using Python/R on iris.csv dataset. 2. Compute Confusion matrix to find TP, FP, TN, FN, Accuracy, Error rate, Precision, Recall the given dataset.

#CODE IMPLEMENTATION

 # 1. Import Libraries

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score, recall_score
```

 # 2. Load Dataset

```
df = pd.read_csv('iris_dataset.csv')
print("First 5 rows:\n", df.head())
print("\nMissing Values:\n", df.isnull().sum())
```

 # 3. Preprocessing

```
X = df.iloc[:, :-1].values   # Features
y = df.iloc[:, -1].values    # Target
```

 # 4. Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
   X, y, test_size=0.3, random_state=42
)
```

 # 5. Train Naïve Bayes Model

```
model = GaussianNB()
model.fit(X_train, y_train)
```

 # 6. Prediction

```
y_pred = model.predict(X_test)
```

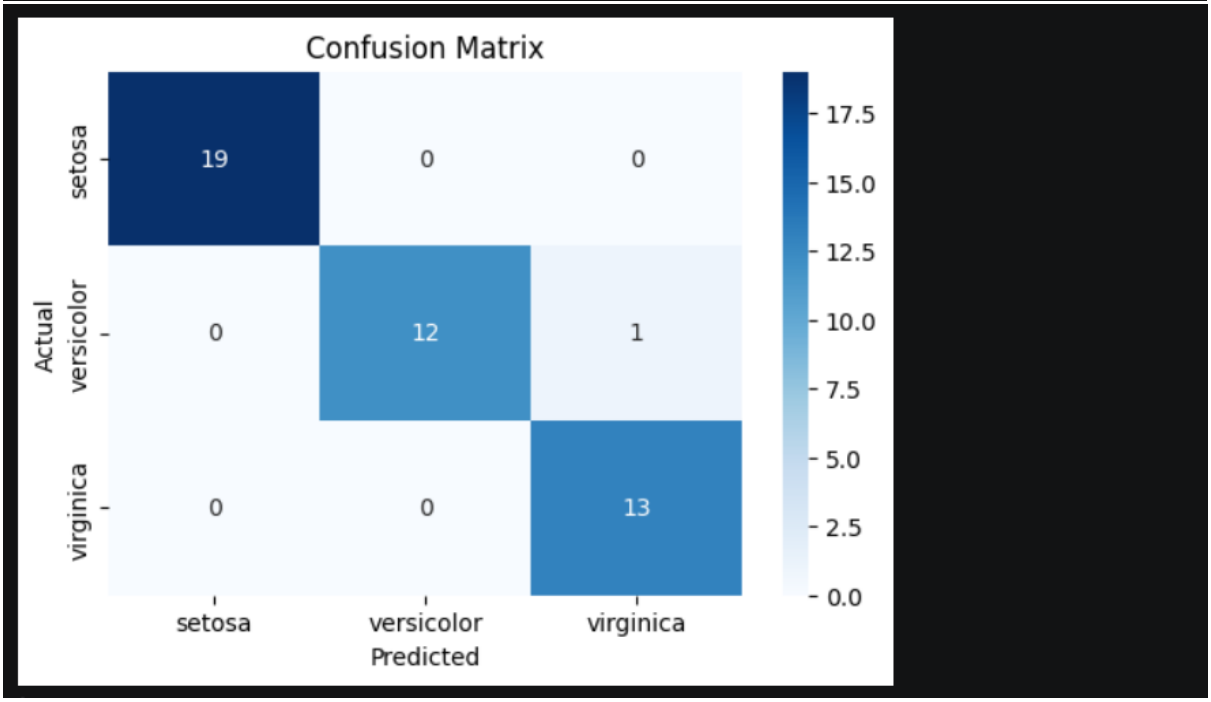
 # 7. Confusion Matrix

```
cm = confusion_matrix(y_test, y_pred)
```

```
print("\nConfusion Matrix:\n", cm)
# 8. Performance Metrics
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, average='weighted')
recall = recall_score(y_test, y_pred, average='weighted')
error_rate = 1 - accuracy
print("\nAccuracy:", accuracy)
print("Precision:", precision)
print("Recall:", recall)
print("Error Rate:", error_rate)
# 9. Confusion Matrix Visualization
plt.figure(figsize=(6,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues',
        xticklabels=model.classes_,
        yticklabels=model.classes_)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

OUTPUT:

```
First 5 rows:
    SepalLength  SepalWidth  PetalLength  PetalWidth Species
0          5.1         3.5          1.4         0.2  setosa
1          4.9         3.0          1.4         0.2  setosa
2          4.7         3.2          1.3         0.2  setosa
3          4.6         3.1          1.5         0.2  setosa
4          5.0         3.6          1.4         0.2  setosa

Missing Values:
 SepalLength    0
SepalWidth     0
PetalLength    0
PetalWidth     0
Species        0
dtype: int64

Confusion Matrix:
 [[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]

Accuracy: 0.9777777777777777
Precision: 0.9793650793650793
Recall: 0.9777777777777777
Error Rate: 0.022222222222222254
```



DATA SCIENCE LABORATORY ASSIGNMENT 7

Text Analytics 1. Extract Sample document and apply following document preprocessing methods: Tokenization, POS Tagging, stop words removal, Stemming and Lemmatization. 2. Create representation of documents by calculating Term Frequency and Inverse DocumentFrequency.

#CODE IMPLEMENTATION

```
# 1. Import Libraries
import nltk
from nltk.corpus import stopwords
from nltk.tokenize import wordpunct_tokenize
from nltk.stem import PorterStemmer
from nltk.stem import WordNetLemmatizer
from sklearn.feature_extraction.text import TfidfVectorizer
# 2. Download Required Data (FIXED)
nltk.download('stopwords')
nltk.download('wordnet')
nltk.download('averaged_perceptron_tagger')
nltk.download('averaged_perceptron_tagger_eng')  # ✅ FIX
# 3. Sample Document
document = """Text Analytics is a crucial step in analyzing unstructured data.
It involves tokenization, lemmatization, and stop words removal."""
# 4. Tokenization
tokens = wordpunct_tokenize(document)
print("\nTokens:\n", tokens)
# 5. POS Tagging (NOW WORKS)
pos_tags = nltk.pos_tag(tokens)
print("\nPOS Tags:\n", pos_tags)
# 6. Stop Words Removal
stop_words = set(stopwords.words('english'))
filtered_tokens = [word for word in tokens if word.lower() not in stop_words]
print("\nFiltered Tokens:\n", filtered_tokens)
```

 # 7. Stemming

```
stemmer = PorterStemmer()

stemmed_words = [stemmer.stem(word) for word in filtered_tokens]

print("\nStemmed Words:\n", stemmed_words)
```

 # 8. Lemmatization

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in filtered_tokens]

print("\nLemmatized Words:\n", lemmatized_words)
```

 # 9. TF-IDF

```
vectorizer = TfidfVectorizer()

tfidf_matrix = vectorizer.fit_transform([document])

print("\nTF-IDF Matrix:\n", tfidf_matrix.toarray())

print("\nFeature Names:\n", vectorizer.get_feature_names_out())
```

OUTPUT:

Tokens:

['Text', 'Analytics', 'is', 'a', 'crucial', 'step', 'in', 'analyzing', 'unstructured', 'data', '.', 'It', 'involves', 'tokenization', ',', 'lemmatization', ',', 'and', 'stop', 'words', 'removal', '.']

POS Tags:

[('Text', 'NN'), ('Analytics', 'NNPS'), ('is', 'VBZ'), ('a', 'DT'), ('crucial', 'JJ'), ('step', 'NN'), ('in', 'IN'), ('analyzing', 'VBG'), ('unstructured', 'JJ'), ('data', 'NNS'), ('.', '.'), ('It', 'PRP'), ('involves', 'VBZ'), ('tokenization', 'NN'), (',', ','), ('lemmatization', 'NN'), (',', ','), ('and', 'CC'), ('stop', 'VB'), ('words', 'NNS'), ('removal', 'NN'), ('.', '.')]

Filtered Tokens:

['Text', 'Analytics', 'crucial', 'step', 'analyzing', 'unstructured', 'data', '.', 'involves', 'tokenization', ',', 'lemmatization', ',', 'stop', 'words', 'removal', '.']

Stemmed Words:

['text', 'analyt', 'crucial', 'step', 'analyz', 'unstructur', 'data', '.', 'involv', 'token', ',', 'lemmat', ',', 'stop', 'word', 'remov', '.']

Lemmatized Words:

['Text', 'Analytics', 'crucial', 'step', 'analyzing', 'unstructured', 'data', '.', 'involves', 'tokenization', ',', 'lemmatization', ',', 'stop', 'word', 'removal', '.']

TF-IDF Matrix:

```
[[0.24253563 0.24253563 0.24253563 0.24253563 0.24253563 0.24253563
 0.24253563 0.24253563 0.24253563 0.24253563 0.24253563 0.24253563
 0.24253563 0.24253563 0.24253563 0.24253563 0.24253563]]
Feature Names:
['analytics' 'analyzing' 'and' 'crucial' 'data' 'in' 'involves' 'is' 'it'
 'lemmatization' 'removal' 'step' 'stop' 'text' 'tokenization'
 'unstructured' 'words']
```

DATA SCIENCE LABORATORY ASSIGNMENT 8

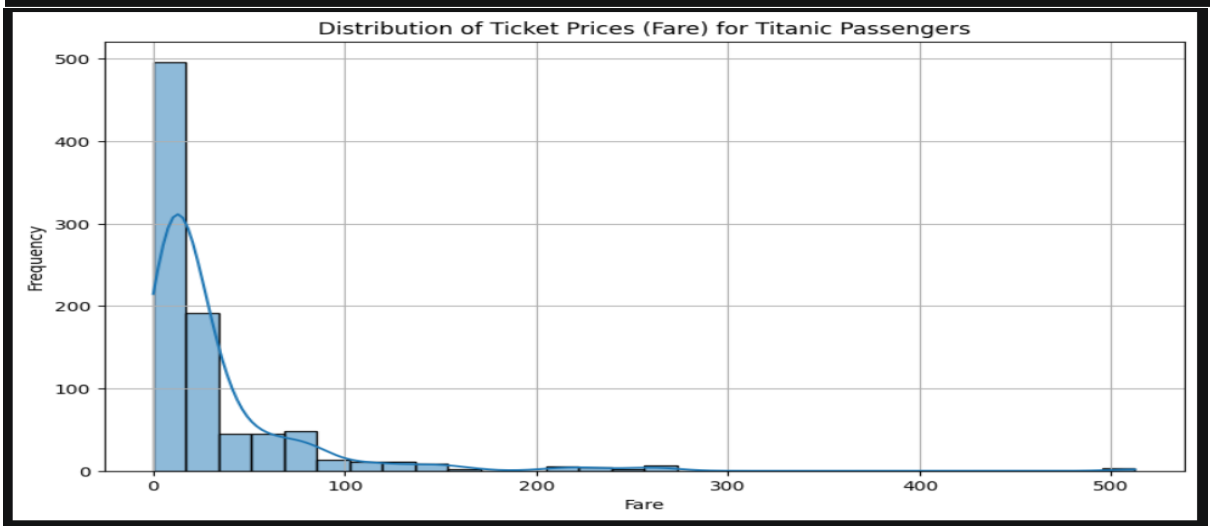
Data Visualization I: 1. Use the inbuilt dataset 'titanic'. The dataset contains 891 rows and contains information about the passengers who boarded the unfortunate Titanic ship. Use the Seaborn library to see if we can find any patterns in the data. 2. Write a code to check how the price of the ticket (column name: 'fare') for each passenger is distributed by plotting a histogram.

#CODE IMPLEMENTATION

```
# 1. Import Libraries
import seaborn as sns
import matplotlib.pyplot as plt
import pandas as pd
# 2. Load Dataset
titanic_data = sns.load_dataset('titanic')
# 3. Explore Dataset
print("\nFirst 5 Rows:\n", titanic_data.head())
print("\nDataset Info:\n")
print(titanic_data.info())
print("\nStatistical Summary:\n", titanic_data.describe())
# 4. Handle Missing Values (important)
# Remove missing fare values (safe method)
titanic_data = titanic_data.dropna(subset=['fare'])
# 5. Plot Histogram of Fare
plt.figure(figsize=(10, 6))
sns.histplot(titanic_data['fare'], kde=True, bins=30)
# 6. Customize Plot
plt.title('Distribution of Ticket Prices (Fare) for Titanic Passengers')
plt.xlabel('Fare')
plt.ylabel('Frequency')
plt.grid(True)
# 7. Show Plot
plt.show()
```

OUTPUT:

```
First 5 Rows:
    survived  pclass     sex   age  sibsp  parch     fare embarked  class  \
0         0       3    male  22.0      1      0   7.2500        S  Third
1         1       1  female  38.0      1      0  71.2833        C  First
2         1       3  female  26.0      0      0   7.9250        S  Third
3         1       1  female  35.0      1      0  53.1000        S  First
4         0       3    male  35.0      0      0   8.0500        S  Third

     who  adult_male deck  embark_town alive  alone
0    man        True  NaN  Southampton    no  False
1  woman       False    C    Cherbourg   yes  False
2  woman       False  NaN  Southampton   yes   True
3  woman       False    C  Southampton   yes  False
4    man        True  NaN  Southampton    no   True

Dataset Info:

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 891 entries, 0 to 890
Data columns (total 15 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   survived     891 non-null    int64
 1   pclass       891 non-null    int64
 2   sex          891 non-null    object
 3   age          714 non-null    float64
 4   sibsp        891 non-null    int64
 5   parch        891 non-null    int64
 6   fare         891 non-null    float64
 7   embarked     889 non-null    object
 8   class        891 non-null    category
 9   who          891 non-null    object
 10  adult_male   891 non-null    bool
 11  deck         203 non-null    category
 12  embark_town  889 non-null    object
 13  alive        891 non-null    object
 14  alone        891 non-null    bool
dtypes: bool(2), category(2), float64(2), int64(4), object(5)
memory usage: 80.7+ KB
None

Statistical Summary:
          survived      pclass         age       sibsp       parch        fare
count  891.000000  891.000000  714.000000  891.000000  891.000000  891.000000
mean     0.383838    2.308642   29.699118    0.523008    0.381594   32.204208
std      0.486592    0.836071   14.526497    1.102743    0.806057   49.693429
min      0.000000    1.000000    0.420000    0.000000    0.000000    0.000000
25%      0.000000    2.000000   20.125000    0.000000    0.000000    7.910400
50%      0.000000    3.000000   28.000000    0.000000    0.000000   14.454200
75%      1.000000    3.000000   38.000000    1.000000    0.000000   31.000000
max      1.000000    3.000000   80.000000    8.000000    6.000000  512.329200
```



DATA SCIENCE LABORATORY ASSIGNMENT 9

Data Visualization II: On Titanic dataset 1. Plot a box plot for distribution of age with respect to each gender along with the information about whether they survived or not. (Column names : 'sex' and 'age') 2. Write observations on the inference from the above statistics.

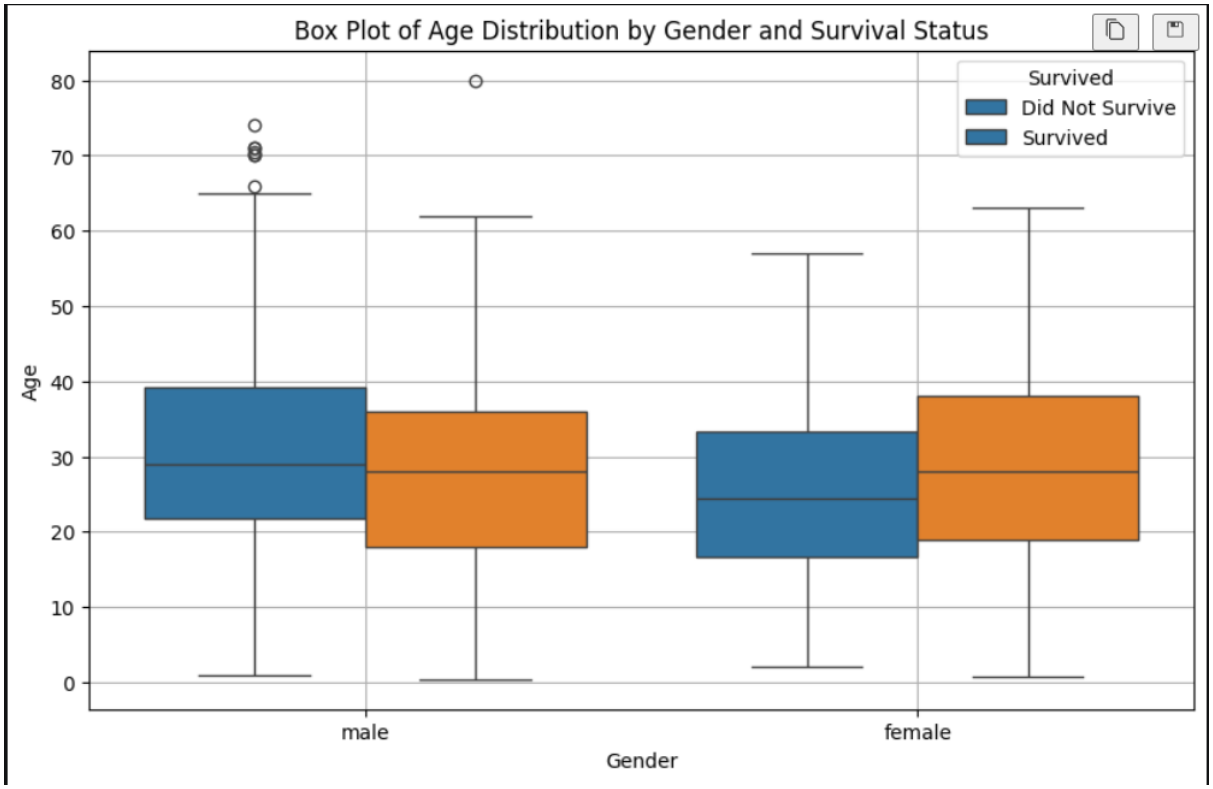
#CODE IMPLEMENTATION

```
# 1. Import Libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# 2. Load Dataset
titanic_data = sns.load_dataset('titanic')
# 3. Data Preprocessing
# Remove missing values in 'age' (important for boxplot)
titanic_data = titanic_data.dropna(subset=['age'])
# 4. Create Box Plot
plt.figure(figsize=(10, 6))
sns.boxplot(
    data=titanic_data,
    x='sex',
    y='age',
    hue='survived'
)
# 5. Customize Plot
plt.title('Box Plot of Age Distribution by Gender and Survival Status')
plt.xlabel('Gender')
plt.ylabel('Age')
plt.legend(
    title='Survived',
    labels=['Did Not Survive', 'Survived']
)
plt.grid(True)
```

 # 6. Show Plot

plt.show()

OUTPUT:



DATA SCIENCE LABORATORY ASSIGNMENT 10

Data Visualization III: On iris dataset give inference as: 1. List down the features and their types (e.g., numeric, nominal) available in the dataset. 2. Create a histogram for each feature in the dataset to illustrate the feature distributions. 3. Create a boxplot for each feature in the dataset. 4. Compare distributions and identify outliers

#CODE IMPLEMENTATION

```
# 1. Import Libraries
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
# 2. Load Dataset
iris_data = pd.read_csv('iris_dataset.csv')
# 3. Explore Dataset
print("First 5 rows:\n", iris_data.head())
print("\nDataset Info:")
print(iris_data.info())
print("\nData Types:\n", iris_data.dtypes)
print("\nStatistical Summary:\n", iris_data.describe())
# 4. Histograms (All Features)
iris_data.hist(bins=10, figsize=(10, 8))
plt.suptitle('Histograms of Iris Dataset Features')
plt.tight_layout()
plt.show()
# 5. Box Plot (All Features)
plt.figure(figsize=(10, 8))
sns.boxplot(data=iris_data.drop('Species', axis=1))
plt.title('Box Plot of Features in Iris Dataset')
plt.show()
# 6. Histograms by Species
plt.figure(figsize=(12, 8))
for i, feature in enumerate(iris_data.columns[:-1]):
```

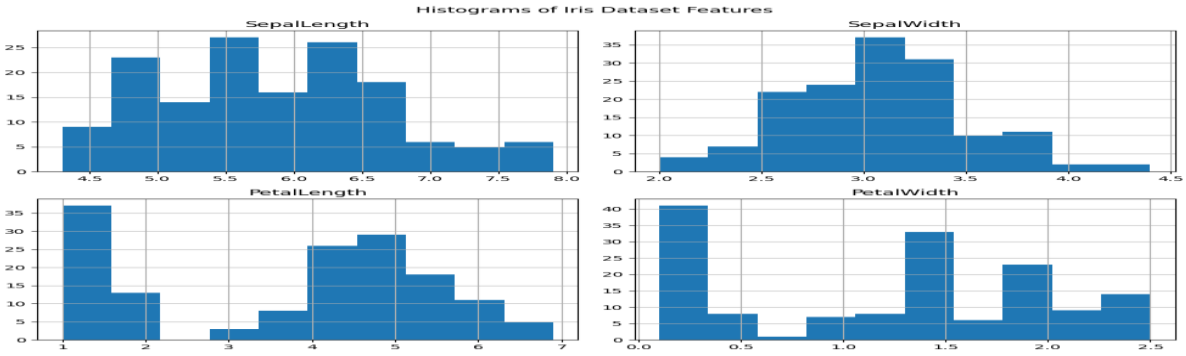
```
    plt.subplot(2, 2, i + 1)
    sns.histplot(data=iris_data, x=feature, hue="Species", kde=True)
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()
```

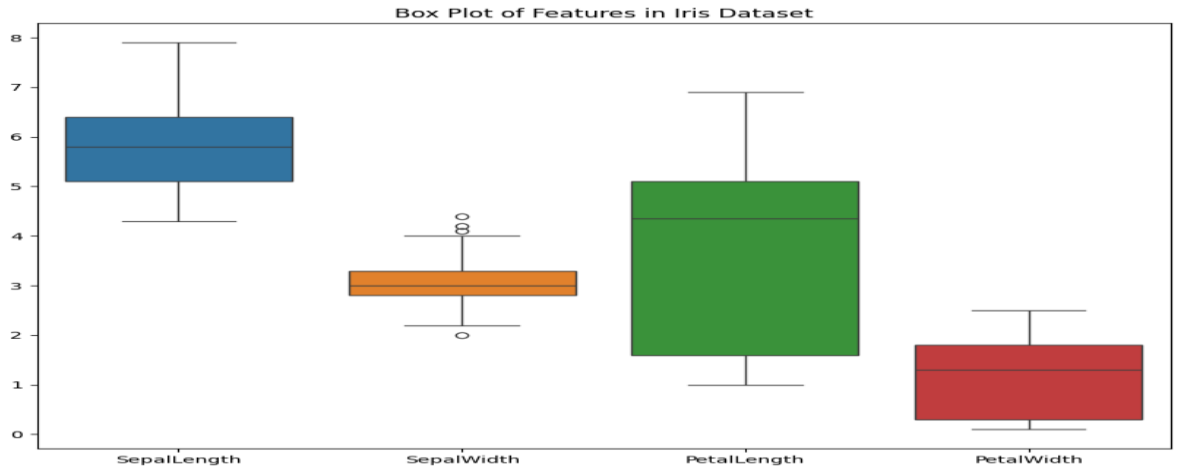
 # 7. Boxplots by Species

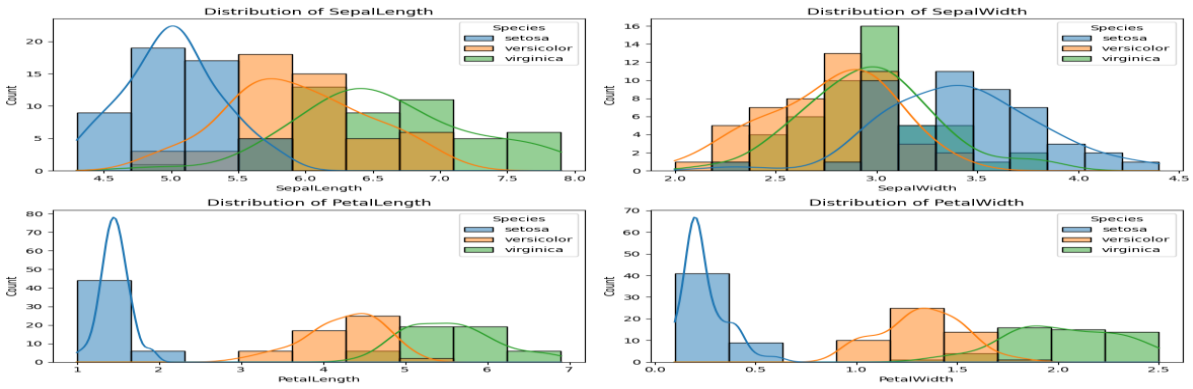
```
plt.figure(figsize=(12, 8))
for i, feature in enumerate(iris_data.columns[:-1]):
    plt.subplot(2, 2, i + 1)
    sns.boxplot(x='Species', y=feature, data=iris_data)
    plt.title(f'Box Plot of {feature} by Species')
plt.tight_layout()
plt.show()
```

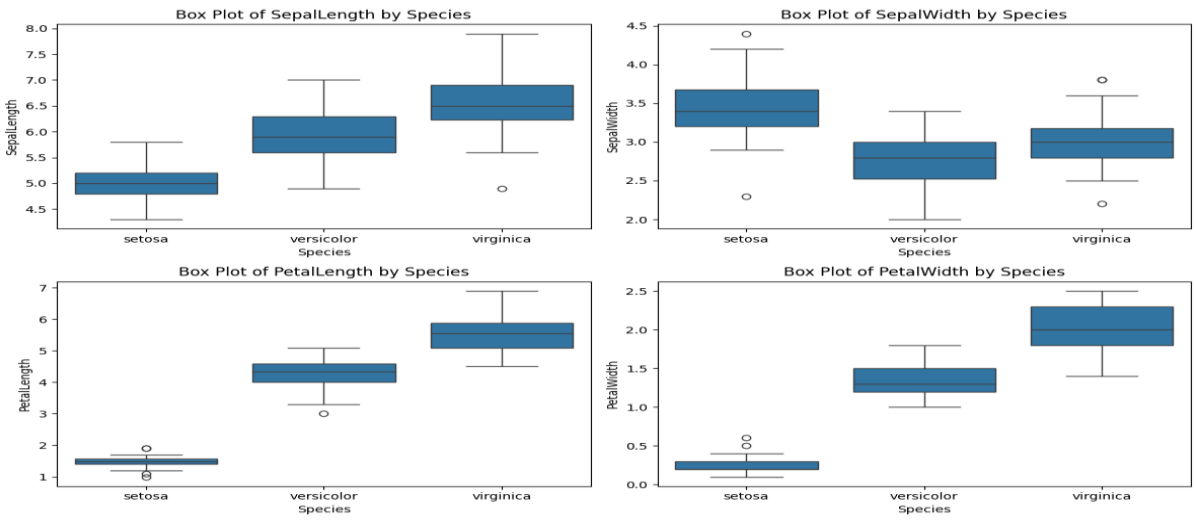
OUTPUT:

```
First 5 rows:
    SepalLength  SepalWidth  PetalLength  PetalWidth Species
0          5.1         3.5          1.4         0.2  setosa
1          4.9         3.0          1.4         0.2  setosa
2          4.7         3.2          1.3         0.2  setosa
3          4.6         3.1          1.5         0.2  setosa
4          5.0         3.6          1.4         0.2  setosa

Dataset Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 150 entries, 0 to 149
Data columns (total 5 columns):
 #   Column       Non-Null Count  Dtype
---  ------       --------------  -----
 0   SepalLength  150 non-null    float64
 1   SepalWidth   150 non-null    float64
 2   PetalLength  150 non-null    float64
 3   PetalWidth   150 non-null    float64
 4   Species      150 non-null    object
dtypes: float64(4), object(1)
memory usage: 6.0+ KB
None

Data Types:
 SepalLength    float64
SepalWidth     float64
PetalLength    float64
PetalWidth     float64
Species         object
dtype: object
       SepalLength  SepalWidth  PetalLength  PetalWidth
count   150.000000  150.000000   150.000000  150.000000
mean      5.843333    3.057333     3.758000    1.199333
std       0.828066    0.435866     1.765298    0.762238
min       4.300000    2.000000     1.000000    0.100000
25%       5.100000    2.800000     1.600000    0.300000
50%       5.800000    3.000000     4.350000    1.300000
75%       6.400000    3.300000     5.100000    1.800000
max       7.900000    4.400000     6.900000    2.500000
```


Histograms of Iris Dataset Features
SepalLength
SepalWidth
PetalLength
PetalWidth


Box Plot of Features in Iris Dataset
SepalLength
SepalWidth
PetalLength
PetalWidth


Distribution of SepalLength
Distribution of SepalWidth
Distribution of PetalLength
Distribution of PetalWidth
Species
setosa
versicolor
virginica
Count


Box Plot of SepalLength by Species
Box Plot of SepalWidth by Species
Box Plot of PetalLength by Species
Box Plot of PetalWidth by Species
setosa
versicolor
virginica
Species

DATA SCIENCE LABORATORY ASSIGNMENT 11

Creating Databases and Tables, Inserting Data, and Running Simple Queries Using Impala

#CODE IMPLEMENTATION

Step 1: Creating a Database – Creating a new database

```
CREATE DATABASE test_db;
```

Step 2: Creating a Table -- Creating a table within the created database

```
 USE test_db;
```

-- Switch to the created database

```
 CREATE TABLE employees ( id INT, name STRING, age INT, department STRING );
```

Step 3: Inserting Data into the Table – Inserting some sample data into the table

```
 INSERT INTO employees VALUES (1, 'Alice', 30, 'HR');
INSERT INTO employees VALUES (2, 'Bob', 35, 'Engineering');
INSERT INTO employees VALUES (3, 'Charlie', 40, 'Marketing');
```

Step 4: Querying Data -- Running a simple query to fetch all records from the table

```
SELECT * FROM employees;
```

-- Query to fetch employees from a specific department

```
 SELECT * FROM employees WHERE department = 'Engineering';
```

Step 5: Dropping the Table and Database (Optional Cleanup) – Dropping the table and database (useful for cleanup)

```
DROP TABLE employees;
DROP DATABASE test_db;
```

OUTPUT:

```
[Impala] > CREATE DATABASE test_db;
Query: CREATE DATABASE test_db
Fetched 0 row(s) in 0.10s

[Impala] > USE test_db;
Query: USE test_db
Fetched 0 row(s) in 0.01s

[Impala] > CREATE TABLE employees (
          id INT,
          name STRING,
          age INT,
          department STRING
        );
Query: CREATE TABLE employees
Fetched 0 row(s) in 0.12s

[Impala] > INSERT INTO employees VALUES (1, 'Alice', 30, 'HR');
Inserted 1 row(s) in 0.20s
[Impala] > INSERT INTO employees VALUES (2, 'Bob', 35, 'Engineering');
Inserted 1 row(s) in 0.18s
[Impala] > INSERT INTO employees VALUES (3, 'Charlie', 40, 'Marketing');
Inserted 1 row(s) in 0.19s

[Impala] > SELECT * FROM employees;
+----+----------+-----+-------------+
| id | name     | age | department  |
+----+----------+-----+-------------+
| 1  | Alice    | 30  | HR          |
| 2  | Bob      | 35  | Engineering |
```

```
| 3  | Charlie  | 40  | Marketing   |
+----+----------+-----+-------------+
Fetched 3 row(s) in 0.15s

[Impala] > SELECT * FROM employees WHERE department = 'Engineering';
+----+------+-----+-------------+
| id | name | age | department  |
+----+------+-----+-------------+
| 2  | Bob  | 35  | Engineering |
+----+------+-----+-------------+
Fetched 1 row(s) in 0.10s

[Impala] > DROP TABLE employees;
Query: DROP TABLE employees
Fetched 0 row(s) in 0.08s

[Impala] > DROP DATABASE test_db;
Query: DROP DATABASE test_db
Fetched 0 row(s) in 0.09s
```

DATA SCIENCE LABORATORY ASSIGNMENT 12

Write a Simple Program in Scala Using Apache Spark Framework

#CODE IMPLEMENTATION

```
//Step 1: Set up your Spark Application
import org.apache.spark.sql.SparkSession
object SparkExample {
  def main(args: Array[String]): Unit = {
// Step 1: Initialize Spark Session
    val spark = SparkSession.builder()
      .appName("Simple Spark Program")
      .master("local")
      .getOrCreate()
// Step 2: Load data from a CSV file into a DataFrame
 val df = spark.read.option("header", "true").csv("people.csv")
// Step 3: Show the first few rows of the DataFrame
    df.show()
// Step 4: Perform a simple transformation and action
    val filtered = df.filter($"age" > 30).select("name", "age")
    println("Filtered Data:")
// Step 5: Show the filtered results
    filtered.show()
// Step 6: Group by and Aggregate Data
    val avgAge = df.groupBy("department").avg("age")
    println("Average Age:")
    avgAge.show()
// Stop the Spark session
    spark.stop()
  }
}
```

OUTPUT:

```
Using Spark's default log4j profile: org/apache/spark/log4j2-defaults.properties
Setting default log level to "WARN".
WARN NativeCodeLoader: Unable to load native-hadoop library for your platform...
Original Data:
+-------+---+-----------+
|   name|age| department|
+-------+---+-----------+
| Alice| 30|         HR|
|   Bob| 35|Engineering|
|Charlie| 40|  Marketing|
| David| 28|Engineering|
|   Eve| 45|         HR|

Filtered Data:
+-------+---+
|   name|age|
+-------+---+
|   Bob| 35|
|Charlie| 40|
|   Eve| 45|
+-------+---+
Average Age:
+-----------+--------+
| department|avg(age)|
+-----------+--------+
|         HR|    37.5|
|Engineering|    31.5|
|  Marketing|    40.0|
+-----------+--------+
INFO SparkContext: Successfully stopped SparkContext
```

DATA SCIENCE LABORATORY ASSIGNMENT 13 (MINIPROJECT)

Develop a Movie Recommendation Model .Using the Scikit-learn Library in Python

#CODE IMPLEMENTATION

#import lilbraries

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

#load dataset

```
df = pd.read_csv("movie.csv")
df.head()
```

 # Data Preprocessing

 # Clean any missing data in the 'genres' column (movie descriptions)

```
df['genres'] = df['genres'].fillna('')
```

 # Extract relevant columns (Movie Title and Description)

```
movies = df[['title', 'genres']]
```

 # Show the first few rows to ensure the data is loaded correctly

```
movies.head()
```

 # TF-IDF Vectorization

```
tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies['genres'])
print(tfidf_matrix.shape)
```

 # Cosine Similarity Calculation

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
```

 # Build a Function for Movie Recommendations

```
indices = pd.Series(movies.index, index=movies['title']).drop_duplicates()
def get_recommendations(title, cosine_sim=cosine_sim):
    idx = indices[title]
    sim_scores = list(enumerate(cosine_sim[idx]))
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)
    sim_scores = sim_scores[1:11]
```

```
    movie_indices = [i[0] for i in sim_scores]
    return movies['title'].iloc[movie_indices]
```

 # Display Recommendations

```
get_recommendations("Balto (1995)")
```

OUTPUT:

```
241                          Gumby: The Movie (1995)
309                        Swan Princess, The (1994)
608                           Aristocats, The (1970)
387     Secret Adventures of Tom Thumb, The (1993)
1007                            Pete's Dragon (1977)
1010                 Fox and the Hound, The (1981)
0                                   Toy Story (1995)
729          Wallace & Gromit: A Close Shave (1995)
580                                 Aladdin (1992)
694                        Oliver & Company (1988)
Name: title, dtype: object
```